

Noise in Quantum Computers (Part 2)

QML

Contents

1	Overview	3
2	Quantum Error Correction Basics	3
2.1	The Idea	3
2.2	The 3-Qubit Bit Flip Code	3
2.3	The 9-Qubit Shor Code	4
2.4	Why QEC is Not Practical Yet	4
3	Zero Noise Extrapolation (ZNE)	5
3.1	The Idea	5
3.2	Noise Amplification Methods	5
3.3	Extrapolation Methods	5
4	Probabilistic Error Cancellation (PEC)	6
4.1	The Idea	6
4.2	PEC in Practice	7
5	Measurement Error Mitigation	7
5.1	The Calibration Matrix Approach	7
5.2	Tensored Mitigation	8
6	Dynamical Decoupling	8
6.1	The Idea	8
6.2	Echo Sequences	8
6.3	Advanced DD Sequences	8
7	Randomized Compiling	9
7.1	The Problem with Coherent Errors	9
7.2	How It Works	9
8	Practical Implications for QML	9
8.1	Circuit Depth Limits	9
8.2	Noise-Aware QML Design	10

8.3	The Sampling Overhead Problem	10
9	Comparing Error Mitigation Techniques	10
10	Noise Characterization Techniques	11
10.1	Quantum State Tomography (QST)	11
10.2	Quantum Process Tomography (QPT)	11
10.3	Randomized Benchmarking (RB)	11
11	Advanced Topic: Twirling	12
12	Summary	12

1 Overview

in part 1 we learned what noise is and how to describe it mathematically. now the question is: what do we do about it? full quantum error correction (QEC) is the long-term answer, but it requires thousands of physical qubits per logical qubit — way beyond what we have now. so for the NISQ era, we rely on quantum error mitigation (QEM): techniques that reduce the impact of noise without the overhead of QEC.

this distinction — mitigation vs correction — is crucial for QML. every variational quantum algorithm running on real hardware today uses some form of error mitigation. understanding these techniques is not optional if you want to do practical QML.

Key Point. error correction vs error mitigation:

- QEC: encodes logical qubits into many physical qubits, detects and corrects errors in real time. requires massive overhead (~ 1000 – 10000 physical qubits per logical qubit). provides fault tolerance.
- QEM: uses classical post-processing to reduce the effect of noise on measurement results. no qubit overhead but requires more measurement shots. does NOT correct individual circuit executions — only improves expectation values statistically.

2 Quantum Error Correction Basics

2.1 The Idea

classical error correction encodes information redundantly. e.g., the repetition code sends $0 \rightarrow 000$ and $1 \rightarrow 111$. if one bit flips, majority vote recovers the original.

quantum error correction does the same thing but with two huge complications:

1. no-cloning: you can't just copy a qubit three times
2. continuous errors: unlike classical bits (which are either flipped or not), quantum errors are continuous (small rotations, partial dephasing, etc.)

Intuition. the key insight of QEC: you don't need to copy quantum information — you need to entangle it across multiple qubits in a way that lets you detect errors without learning the encoded state. this is possible because of the structure of quantum error channels (they can be discretized into Pauli errors).

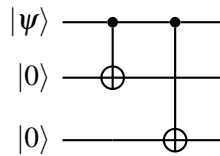
2.2 The 3-Qubit Bit Flip Code

Definition 2.1. the simplest quantum error correcting code encodes 1 logical qubit into 3 physical qubits:

$$|0\rangle_L = |000\rangle, \quad |1\rangle_L = |111\rangle$$

a general logical state $\alpha|0\rangle_L + \beta|1\rangle_L = \alpha|000\rangle + \beta|111\rangle$.

encoding circuit:



this code can correct a single bit flip (X error) on any one of the three qubits.

syndrome measurement: measure the parity of qubit pairs without measuring the qubits themselves:

- measure Z_1Z_2 (parity of qubits 1 and 2)
- measure Z_2Z_3 (parity of qubits 2 and 3)

error	Z_1Z_2	Z_2Z_3	correction
none	+1	+1	none
X_1	-1	+1	apply X_1
X_2	-1	-1	apply X_2
X_3	+1	-1	apply X_3

Key Point. the syndrome measurement reveals which qubit had an error without revealing what state the qubit is in. this is the magic of QEC — u extract error information without collapsing the encoded quantum state. the parity measurements r “stabilizers” and this leads to the stabilizer formalism for QEC.

2.3 The 9-Qubit Shor Code

the bit flip code only handles X errors. the phase flip code (similar structure but in the hadamard basis) only handles Z errors. shor’s 9-qubit code concatenates both to handle arbitrary single-qubit errors:

$$|0\rangle_L = \frac{1}{2\sqrt{2}}(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)$$

Intuition. any single-qubit error can be decomposed into pauli errors (I, X, Y, Z). the inner repetition code handles X errors, the outer phase code handles Z errors, and $Y = iXZ$ is handled by both. this discretization of errors into paulis is why QEC works despite errors being continuous.

2.4 Why QEC is Not Practical Yet

Key Point. current state of QEC:

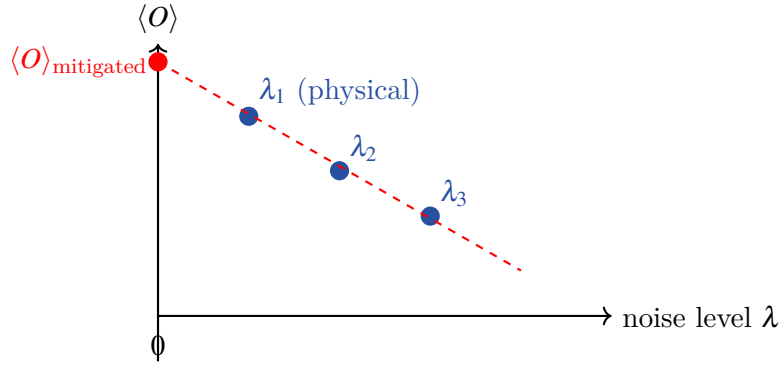
- the surface code is the leading QEC architecture. needs $\sim d^2$ physical qubits per logical qubit (where d is the code distance) plus ancillas for syndrome measurement
- for useful error rates: $d \sim 20\text{--}30$, meaning $\sim 1000\text{--}3000$ physical qubits per logical qubit
- a useful quantum computer might need ~ 100 logical qubits $= \sim 100,000\text{--}300,000$ physical qubits
- current hardware: $\sim 100\text{--}1000$ physical qubits (IBM Eagle/Heron, Google Sycamore)
- we r at least 5–10 years from fault-tolerant QEC at scale
- until then: NISQ devices + error mitigation

3 Zero Noise Extrapolation (ZNE)

3.1 The Idea

ZNE is the simplest and most widely used error mitigation technique. the idea:

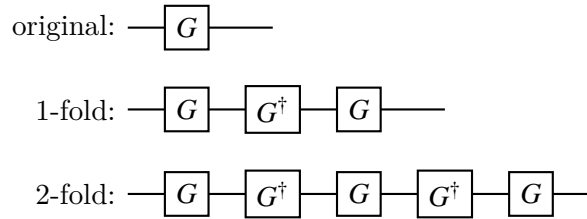
1. run ur circuit at the physical noise level and get a noisy result
2. artificially increase the noise (by stretching gates or inserting identity pairs) and run the noisier circuit
3. extrapolate back to what the result would be at zero noise



3.2 Noise Amplification Methods

method 1: pulse stretching. physically stretch the gate pulses by a factor $c > 1$. this increases the time the qubit is exposed to decoherence, effectively scaling the noise by c .

method 2: unitary folding. replace each gate G with $G \cdot G^\dagger \cdot G$ (which is mathematically identical to G but takes 3x the circuit depth). more generally, fold n times: $G(G^\dagger G)^n$. the noise level scales roughly as $(2n+1) \times$ the original noise.



3.3 Extrapolation Methods

Definition 3.1. given noisy expectation values at noise levels $\lambda_1, \lambda_2, \dots, \lambda_k$, extrapolate to $\lambda = 0$:
linear extrapolation (Richardson, 2 noise levels):

$$\langle O \rangle_{\text{mitigated}} = \frac{c\langle O \rangle_{\lambda_1} - \langle O \rangle_{c\lambda_1}}{c - 1}$$

polynomial extrapolation (k noise levels): fit a degree- $(k-1)$ polynomial through the k data points and evaluate at $\lambda = 0$.

exponential extrapolation: fit $\langle O \rangle(\lambda) = ae^{-b\lambda} + c$ and evaluate at $\lambda = 0$. often more physically motivated for depolarizing noise.

Example 3.1. linear extrapolation with noise amplification factor $c = 3$:

suppose $\langle O \rangle_\lambda = 0.6$ (physical noise) and $\langle O \rangle_{3\lambda} = 0.2$ (3x noise).

$$\langle O \rangle_{\text{mitigated}} = \frac{3 \times 0.6 - 0.2}{3 - 1} = \frac{1.8 - 0.2}{2} = 0.8$$

if the true noiseless value is 0.85, weve gone from an error of 0.25 to an error of 0.05.

Key Point. ZNE tradeoffs:

- pros: simple to implement, no knowledge of the noise model needed, works with any circuit
- cons: increases variance (need more shots), assumes noise scales predictably, extrapolation can be inaccurate for complex noise models
- shot overhead: roughly $O(c^2)$ more shots needed to maintain the same statistical precision at noise level $c\lambda$

4 Probabilistic Error Cancellation (PEC)

4.1 The Idea

PEC is more principled than ZNE but requires knowledge of the noise model. the idea:

1. characterize the noise channel $\mathcal{E}$ for each gate (via tomography)
2. express the ideal gate as a linear combination of noisy implementable operations:

$$\mathcal{U}_{\text{ideal}} = \sum_i \eta_i \mathcal{O}_i$$

where $\mathcal{O}_i$ r operations u can actually run on hardware and η_i r real coefficients (can be negative)

3. sample from this decomposition probabilistically and correct using importance sampling

Definition 4.1. the quasi-probability decomposition of the ideal channel $\mathcal{U}$ given noisy channel $\mathcal{E}$:

$$\mathcal{U} = \sum_i \eta_i \mathcal{B}_i$$

where $\mathcal{B}_i$ r “basis operations” (noisy gates followed by additional pauli gates). the coefficients η_i can be negative (hence “quasi-probability”).

Key Point. PEC properties:

- unbiased: gives the exact noiseless answer in expectation (no extrapolation assumptions)
- cost: the sampling overhead is γ^2 where $\gamma = \sum_i |\eta_i|$ is the 1-norm of the quasi-probability distribution. γ grows exponentially with circuit depth for general noise.
- requirement: need a good characterization of the noise model for each gate

for a circuit with d noisy gates, each with per-gate overhead γ_i : total overhead = $\prod_i \gamma_i^2$. if $\gamma_i \approx 1 + \varepsilon$, total overhead $\approx e^{2d\varepsilon}$. exponential in circuit depth.

4.2 PEC in Practice

Example 4.1. suppose the ideal gate is $R_z(\theta)$ but the actual implementation has a small depolarizing error:

$$\mathcal{E}(\rho) = (1-p)R_z(\theta)\rho R_z(\theta)^\dagger + \frac{p}{3}(XR_z(\theta)\rho R_z(\theta)^\dagger X + Y\dots + Z\dots)$$

the quasi-probability decomposition is:

$$\mathcal{U}_{\text{ideal}} = \frac{1}{1-p}\mathcal{E} - \frac{p}{3(1-p)}(\mathcal{E} \circ \mathcal{X} + \mathcal{E} \circ \mathcal{Y} + \mathcal{E} \circ \mathcal{Z})$$

where $\mathcal{X}(\rho) = X\rho X$, etc. the 1-norm overhead: $\gamma = \frac{1+p}{1-p} \approx 1 + 2p$ for small p .

5 Measurement Error Mitigation

5.1 The Calibration Matrix Approach

measurement errors r often the easiest to characterize and mitigate. the idea:

1. prepare each computational basis state $|x\rangle$ (e.g., $|00\rangle, |01\rangle, |10\rangle, |11\rangle$)
2. measure many times to build a calibration matrix A where $A_{ij} = P(\text{measure } i | \text{prepared } j)$
3. given noisy measurement results $\vec{p}_{\text{noisy}}$, recover the ideal results: $\vec{p}_{\text{ideal}} = A^{-1}\vec{p}_{\text{noisy}}$

Definition 5.1. the calibration matrix for n qubits is a $2^n \times 2^n$ matrix:

$$A_{ij} = P(\text{measure outcome } i | \text{state is } j)$$

each column sums to 1 (its a stochastic matrix). for perfect measurements, $A = I$.

Example 5.1. for 2 qubits, prepare and measure each of $|00\rangle, |01\rangle, |10\rangle, |11\rangle$:

$$A = \begin{pmatrix} 0.95 & 0.02 & 0.03 & 0.01 \\ 0.03 & 0.93 & 0.01 & 0.04 \\ 0.01 & 0.03 & 0.94 & 0.02 \\ 0.01 & 0.02 & 0.02 & 0.93 \end{pmatrix}$$

if noisy measurement gives $\vec{p}_{\text{noisy}} = (0.45, 0.05, 0.05, 0.45)$:

$$\vec{p}_{\text{ideal}} \approx A^{-1}\vec{p}_{\text{noisy}} \approx (0.47, 0.03, 0.03, 0.47)$$

Key Point. practical issues with measurement mitigation:

- the calibration matrix is $2^n \times 2^n$ — exponential in the number of qubits. for $n > 10$, u cant even store it
- solution: assume measurement errors r independent (or correlated only between neighbors). then the calibration matrix is a tensor product of small matrices: $A = A_1 \otimes A_2 \otimes \dots$
- A^{-1} can produce negative probabilities — need constrained optimization (least squares with non-negativity constraints)
- calibration drifts over time — need to recalibrate regularly

5.2 Tensored Mitigation

for n qubits with independent measurement errors, each qubit i has a 2×2 calibration matrix:

$$A_i = \begin{pmatrix} 1 - p_{0|1}^{(i)} & p_{0|1}^{(i)} \\ p_{1|0}^{(i)} & 1 - p_{1|0}^{(i)} \end{pmatrix}$$

the full calibration matrix is $A = A_1 \otimes A_2 \otimes \dots \otimes A_n$ and its inverse is $A^{-1} = A_1^{-1} \otimes \dots \otimes A_n^{-1}$. this only requires $2n$ calibration circuits instead of 2^n .

6 Dynamical Decoupling

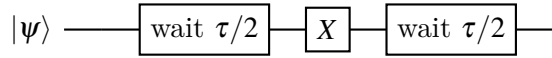
6.1 The Idea

Definition 6.1. dynamical decoupling (DD) is a technique to suppress decoherence during idle periods by applying fast pulses that “average out” the environment’s effect.

when a qubit is idle (waiting for other operations), it decoheres. DD fights this by inserting sequences of pulses that refocus the qubit’s evolution.

6.2 Echo Sequences

the simplest DD sequence is the spin echo (Hahn echo):



Intuition. heres how it works. during the first wait period, the qubit picks up a random phase ϕ from environmental noise: $|\psi\rangle \rightarrow e^{i\phi Z} |\psi\rangle$. the X pulse flips the qubit, effectively negating the phase. during the second wait period, the qubit picks up approximately the same phase ϕ again (assuming the noise is slow compared to τ). but now it adds to $-\phi$, canceling out:

$$e^{i\phi Z} \cdot X \cdot e^{i\phi Z} |\psi\rangle \approx X |\psi\rangle \text{ (undone by a final } X \text{ or tracked in software).}$$

the net effect: low-frequency noise is canceled.

6.3 Advanced DD Sequences

more sophisticated sequences suppress higher-order noise:

CPMG (Carr-Purcell-Meiboom-Gill): $\tau - X - 2\tau - X - 2\tau - X - \dots - X - \tau$

XY4: alternates X and Y pulses: $X - Y - X - Y$. suppresses noise in all directions.

Uhrig DD (UDD): optimally placed pulses (not equally spaced) for specific noise spectra.

Key Point. DD on real hardware:

- IBM Quantum automatically applies DD during idle periods in circuits
- typical improvement: 2x–5x reduction in decoherence during idle times
- limitations: pulses themselves r imperfect and add small errors; doesnt help during active gates
- cost: zero qubit overhead, minimal circuit depth overhead (just extra single-qubit gates)

7 Randomized Compiling

7.1 The Problem with Coherent Errors

coherent errors (systematic over/under-rotation of gates) r particularly nasty bcs they accumulate quadratically with circuit depth:

$$\text{coherent error after } d \text{ gates} \sim (d\epsilon)^2$$

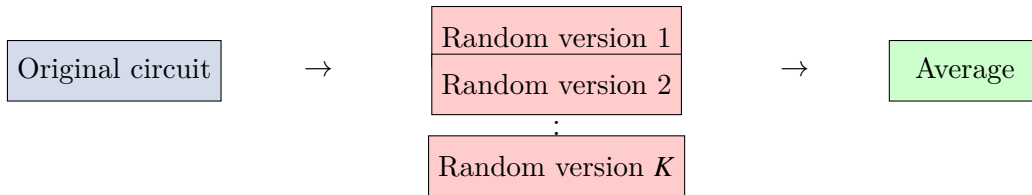
compared to incoherent (random) errors which accumulate linearly: $\sim d\epsilon$.

Definition 7.1. randomized compiling (RC) converts coherent errors into incoherent errors by randomly “twirling” each gate. the idea: insert random pauli gates before and after each gate, choosing the paulis so that the ideal circuit is unchanged but the coherent errors r randomized.

7.2 How It Works

for each gate G in the circuit:

1. randomly choose pauli gates P_{before} and compute $P_{\text{after}} = G \cdot P_{\text{before}} \cdot G^\dagger$ (so that the ideal circuit is $P_{\text{after}} \cdot G \cdot P_{\text{before}} = G$)
2. absorb P_{before} into the preceding gate and P_{after} into the following gate
3. run many randomized versions and average the results



Key Point. randomized compiling benefits:

- converts coherent errors (quadratic accumulation) to incoherent errors (linear accumulation)
- the effective noise channel becomes a pauli channel (much easier to analyze and mitigate with ZNE or PEC)
- minimal overhead: just extra single-qubit paulis (fast, high-fidelity gates)
- often combined with ZNE or PEC for maximum benefit

8 Practical Implications for QML

8.1 Circuit Depth Limits

Key Point. noise fundamentally limits wt QML can do on NISQ hardware:

1. circuit depth: with 2-qubit gate fidelity $\sim 99\%$, useful circuit depth is $\sim 50\text{--}100$ gates. this limits the expressibility of quantum models.

2. number of qubits: more qubits = more noise sources = faster decoherence. current useful range: $\sim 10\text{--}50$ qubits.
3. number of shots: error mitigation techniques (especially PEC) require exponentially more shots as circuit depth increases. this limits practical circuit depth even further.
4. parameter optimization: noisy gradients make training harder. noise-induced barren plateaus can make the loss landscape flat.

8.2 Noise-Aware QML Design

Key Point. strategies for building QML models that work on noisy hardware:

- use hardware-efficient ansatzes — circuits that match the native gate set and connectivity of the hardware
- keep circuits shallow — fewer gates means less noise
- use local cost functions — they are less affected by noise than global cost functions (Cerezo et al. 2021)
- apply measurement error mitigation — it's cheap and always helps
- use ZNE for expectation value estimation in variational circuits
- consider error-robust ansatzes — circuit designs that are naturally less sensitive to certain error types
- noise-aware training: include noise models in the classical simulation during hyperparameter tuning

8.3 The Sampling Overhead Problem

Intuition. here's the fundamental tension in NISQ QML:

expressibility needs deep circuits $\rightarrow$ more noise $\rightarrow$ need more error mitigation $\rightarrow$ need exponentially more measurement shots $\rightarrow$ kills any quantum advantage.

noise robustness needs shallow circuits $\rightarrow$ limited expressibility $\rightarrow$ might not be more powerful than classical models.

this is the NISQ dilemma. finding the sweet spot — circuits deep enough to be useful but shallow enough to be mitigatable — is the central challenge of practical QML.

9 Comparing Error Mitigation Techniques

technique	bias	overhead	noise model	complexity
ZNE	biased	moderate	not needed	easy
PEC	unbiased	exponential	required	moderate
meas. mitigation	unbiased	linear	partial	easy
dynamical decoupling	N/A	minimal	not needed	easy
randomized compiling	biased	moderate	not needed	moderate

Key Point. in practice, the best approach combines multiple techniques:

1. randomized compiling: tailors coherent errors into incoherent ones (apply always)
2. dynamical decoupling: during idle times (apply always, near-free)
3. measurement error mitigation: on the readout (apply always, cheap)
4. ZNE or PEC: on the expectation values (apply when u can afford the shot overhead)

this layered approach gives the best results on current hardware.

10 Noise Characterization Techniques

before u can mitigate noise, u need to know wt the noise is. here r the main characterization techniques:

10.1 Quantum State Tomography (QST)

Definition 10.1. QST fully reconstructs the density matrix of an n -qubit state by measuring in multiple bases. for n qubits, u need at least 3^n measurement settings (all combinations of X, Y, Z measurements on each qubit) and $4^n - 1$ parameters to estimate.

QST is exponentially expensive — only practical for $\lesssim 10$ qubits. but its the gold standard for verifying small quantum states.

10.2 Quantum Process Tomography (QPT)

Definition 10.2. QPT fully characterizes a quantum channel by preparing a complete set of input states, sending them through the channel, and doing QST on each output. for n qubits: prepare 4^n input states, do QST on each $\rightarrow O(16^n)$ total experiments.

even more expensive than QST. only practical for 1–2 qubit gates.

10.3 Randomized Benchmarking (RB)

Definition 10.3. randomized benchmarking estimates the average gate fidelity without full process tomography:

1. apply a random sequence of m clifford gates
2. apply the inverse of the entire sequence
3. measure the probability of returning to the initial state
4. repeat for various sequence lengths m
5. fit the decay: $F(m) = A \cdot p^m + B$
6. the average error per gate: $r = (1 - p)(1 - 1/2^n)$

Intuition. RB is efficient bcs it only estimates a single number (average fidelity) rather than the full process matrix. the random clifford twirling converts any noise into depolarizing noise, and the depolarizing parameter decays exponentially with sequence length. the decay rate tells u the average error per gate.

RB is wt hardware vendors use to report gate fidelities. when IBM says their CNOT fidelity is 99.5%, thats from RB.

11 Advanced Topic: Twirling

Definition 11.1. twirling is a technique that simplifies a noise channel by averaging over a group of unitaries:

$$\mathcal{E}_{\text{twirled}}(\rho) = \frac{1}{|G|} \sum_{g \in G} g^\dagger \mathcal{E}(g \rho g^\dagger) g$$

pauli twirling: average over the pauli group. converts any noise channel into a pauli channel (only diagonal elements survive in the pauli transfer matrix).

clifford twirling: average over the clifford group. converts any noise channel into a depolarizing channel.

Intuition. twirling is the theoretical foundation behind both randomized benchmarking and randomized compiling. by averaging over random unitaries, u “symmetrize” the noise into a simpler form. pauli twirling gives u a pauli channel (easy to work with). clifford twirling gives u depolarizing noise (the simplest possible noise model).

this is deeply connected to representation theory — the twirled channel lives in the commutant of the twirling group, and for the pauli/clifford groups, this commutant has a simple structure.

12 Summary

Key Point. key takeaways from both noise lectures:

1. QEC is the ultimate solution but requires $\sim 1000x$ qubit overhead. not practical yet.
2. for NISQ: use error mitigation techniques (ZNE, PEC, measurement mitigation, DD, RC)
3. ZNE: amplify noise, extrapolate to zero. simple but biased.
4. PEC: unbiased but exponential sampling overhead. needs noise characterization.
5. measurement mitigation: calibration matrix approach. cheap and effective.
6. dynamical decoupling: free lunch for idle qubits. always use it.
7. randomized compiling: converts coherent errors to incoherent. always helps.
8. the NISQ dilemma: need deep circuits for expressibility, but noise kills deep circuits. finding the sweet spot is the central challenge.
9. for QML specifically: noise limits circuit depth, adds training difficulty, and can create noise-induced barren plateaus. noise-aware design is essential.

with the noise story complete, were ready to move on to the ML side of things. next: classical machine learning foundations.